Solution: Row With Maximum Ones(easy)

Problem Statement

Given a binary matrix that has dimensions $m * n$, consisting of ones and zeros. Determine the row that contains the highest number of ones and return two values: the zero-based index of this row and the actual count of ones it possesses.

If there is a tie, i.e., multiple rows contain the same maximum number of ones, we must select the row with the lowest index.

Examples

Example 1:

- **Input**: `[[1, 0], [1, 1], [0, 1]]`
- **Expected Output**: `[1, 2]`
- **Justification**: The second row `[1, 1]` contains the most ones, so the output is `[1, 2]`.

Example 2:

- **Input**: `[[0, 1, 1], [0, 1, 1], [1, 1, 1]]`
- **Expected Output**: `[2, 3]`
- **Justification**: The third row `[1, 1, 1]` has the most ones, leading to the output `[2, 3]`.

Example 3:

- **Input**: `[[1, 0, 1], [0, 0, 1], [1, 1, 0]]`
- **Expected Output**: `[0, 2]`
- **Justification**: Both the first and third rows contain two ones, but we choose the first due to its lower index, resulting in `[0, 2]`.

Solution

The fundamental task is to identify the row with the maximum number of ones and simultaneously track the number of ones in this row.

Solution Overview

To solve this problem, we will systematically examine each row of a given binary matrix to identify the row with the highest number of 1s. The solution involves iterating over each row, counting the 1s present, and comparing this count with the highest number of 1s recorded so far. We maintain two variables: one for the highest count of 1s encountered, and another for the index of the row where this occurred. This method ensures we check every element in the matrix precisely once, leading to an efficient approach in determining the row that contains the maximum number of 1s.

Here is the algorithm with detailed steps.

1. **Initialize Variables**: Set up two variables: `maxOnesCount`, initially 0, for tracking the maximum number of 1s found in any row; and `maxOnesIdx`, initially -1, for storing the index of the row with this maximum count.

2. **Iterate Through Each Row**: Loop through each row of the matrix. For every row, perform the following steps:

   a. **Count 1s in the Current Row**: Initialize a counter for the number of 1s in this row. Traverse through each element of the row, incrementing the counter for each 1 encountered.

   b. **Update Maximum Count and Row Index**: If the count of 1s in the current row exceeds `maxOnesCount`, update `maxOnesCount` with the new count and set `maxOnesIdx` to the current row's index.

3. **Handling No 1s Scenario**: If no 1s are found in the entire matrix, `maxOnesIdx` remains -1. This indicates that there's no row with 1s.

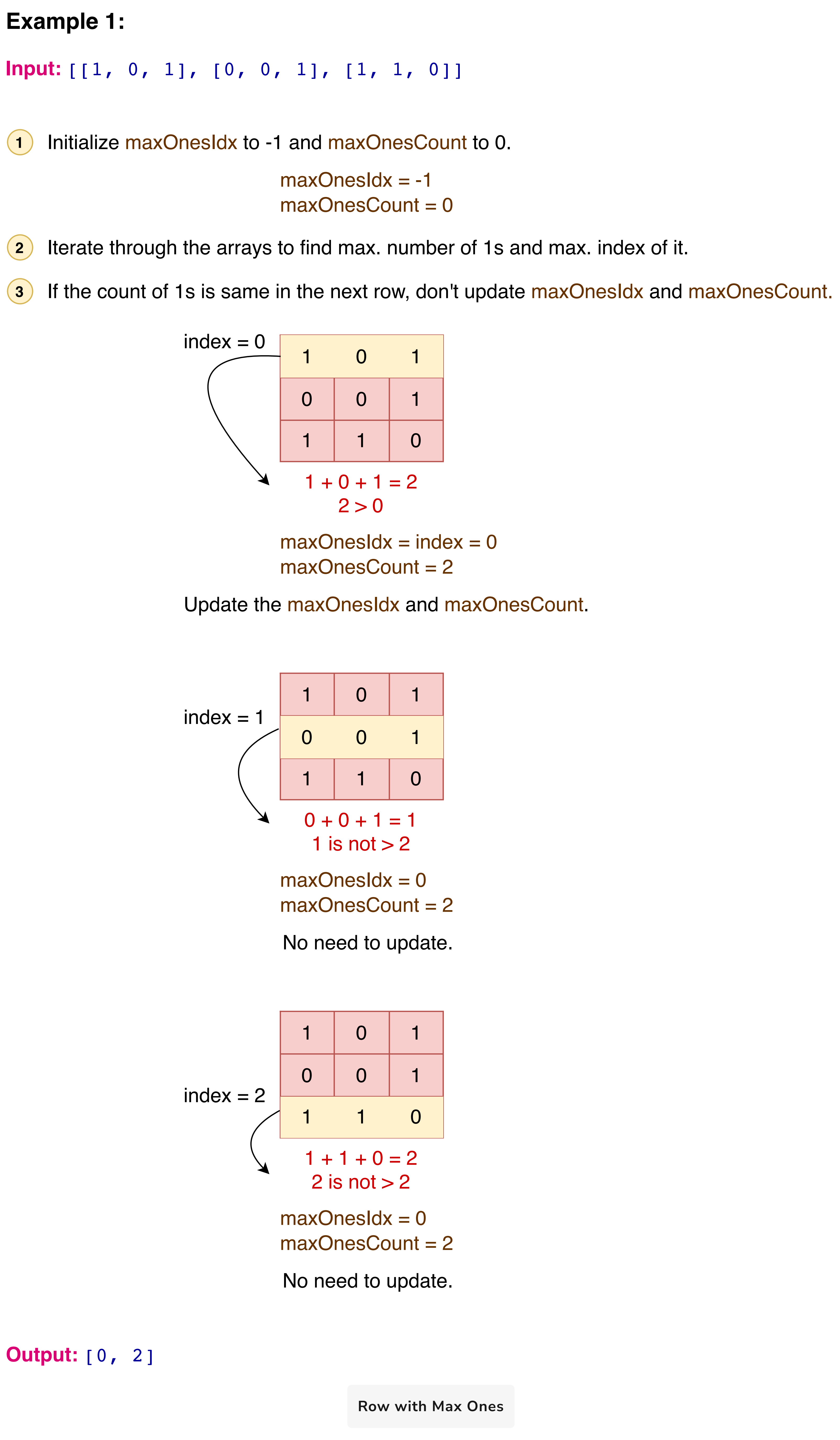
4. **Return the Result**: At the end of the iteration, return `[maxOnesIdx, maxOnesCount]`.

Algorithm Walkthrough

Given the input `[[1, 0, 1], [0, 0, 1], [1, 1, 0]]`:

- Initialize `maxOnesIdx` to `-1` and `maxOnesCount` to `0`.
- Loop through each row:
  - Row 0 `[1, 0, 1]`: Count of ones = 2. It is greater than `maxOnesCount` so update `maxOnesIdx` to `0` and `maxOnesCount` to `2`.
  - Row 1 `[0, 0, 1]`: Count of ones = 1. It is not greater than `maxOnesCount`, no update occurs.
  - Row 2 `[1, 1, 0]`: Count of ones = 2. It is equal to `maxOnesCount`, so no update occurs to maintain the smallest index.
- Return `[maxOnesIdx, maxOnesCount]` which yields `[0, 2]`.

Here is the visual representation of the algorithm:



Example 1:

Input: `[[1, 0, 1], [0, 0, 1], [1, 1, 0]]`

① Initialize maxOnesIdx to -1 and maxOnesCount to 0.

maxOnesIdx = -1
maxOnesCount = 0

② Iterate through the arrays to find max. number of 1s and max. index of it.

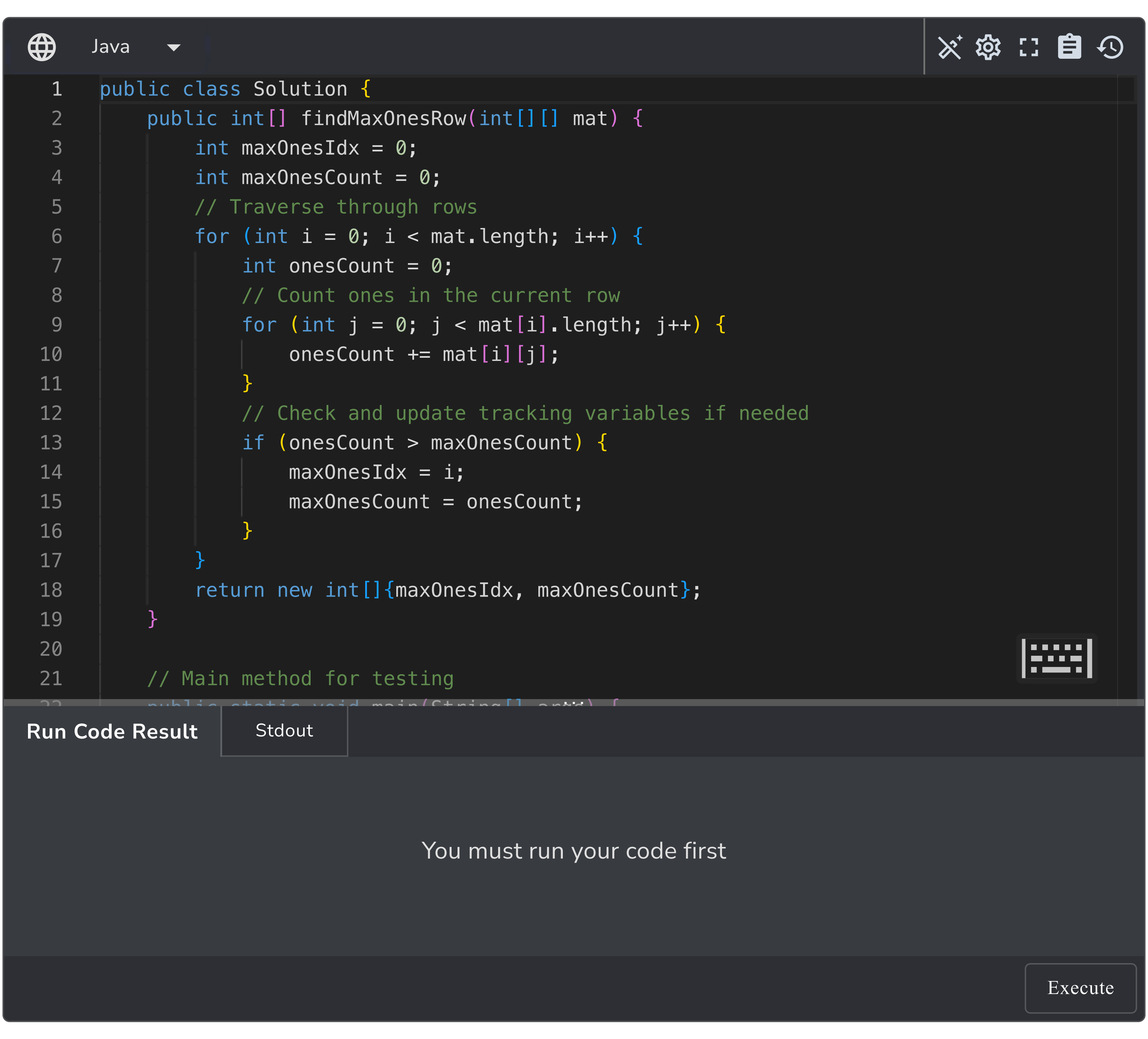
③ If the count of 1s is same in the next row, don't update maxOnesIdx and maxOnesCount.

Output: `[0, 2]`

Row with Max Ones

Code

Here is the code for this algorithm:



```java
public class Solution {
    public int[] findMaxOnesRow(int[][] mat) {
        int maxOnesIdx = 0;
        int maxOnesCount = 0;
        // Traverse through rows
        for (int i = 0; i < mat.length; i++) {
            int onesCount = 0;
            // Count ones in the current row
            for (int j = 0; j < mat[i].length; j++) {
                onesCount += mat[i][j];
            }
            // Check and update tracking variables if needed
            if (onesCount > maxOnesCount) {
                maxOnesIdx = i;
                maxOnesCount = onesCount;
            }
        }
        return new int[]{maxOnesIdx, maxOnesCount};
    }

    // Main method for testing
```

You must run your code first

Time Complexity

- We iterate through each row of the matrix to find the sum of ones in that row. If there are "m" rows and "n" columns, this operation takes O(m*n) time.
- We are also keeping track of the maximum count of ones and its corresponding row index while iterating through the matrix, which takes O(1) time for each row.
- Therefore, the overall time complexity of the algorithm is O(m*n).

Space Complexity

- We are using only a constant amount of extra space to store the maximum count of ones and its corresponding row index.
- The input matrix is given by the problem, and we are not using any additional space that grows with the input.
- Thus, the space complexity of the algorithm is O(1).

← Previous
Problem 3: Row With Maximum Ones(Easy)

Next →
Introduction To Stack

☐ Mark as Completed